UNIT IV: Clustering

Clustering is an **unsupervised ML technique** that involves grouping a set of data points into clusters such that points in the same cluster are **more similar** to each other than to those in other clusters. It is used to **discover inherent structures or patterns** in data without predefined labels.

- Groups data based on similarity/distance measures.
- Does not require labeled data.
- Helps in summarizing and understanding large datasets.

Partitioning Clustering is a type of **unsupervised learning** method where the dataset is divided into a set of **non-overlapping subsets or clusters**, such that each data point belongs to exactly one cluster.

Feature	Description
Type	Flat clustering (non-hierarchical)
No of clusters (k)	Must be specified before the algorithm starts
Data point membership	Each data point belongs to exactly one cluster
Objective	Minimize intra-cluster distance and maximize inter-cluster distance
Common Algorithms	K-Means, K-Medoids, CLARANS

Working of Partitioning Clustering (Using K-Means as an Example):
1. **Initialization**: Randomly select k data points as initial centroids.
2. **Assignment Step**: Assign point to nearest centroid, forming k clusters.
3. **Update Step**: Compute centroids by taking mean of points in a cluster.
4. **Repeat**: Repeat assignment and update steps until convergence.

Advantages: Simple and fast for small to medium-sized datasets, efficient when clusters are spherical and of similar size.
Disadvantages: Requires prior knowledge of k (number of clusters), Sensitive to the initial choice of centroids. Not suitable for clusters with complex shapes or varying sizes.

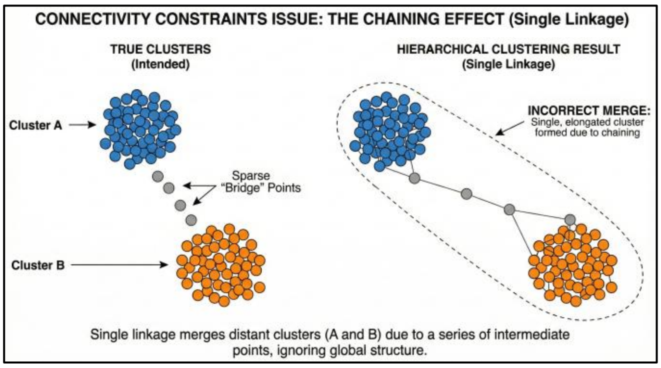
Criteria	Hierarchical Clustering	K-Means Clustering
Type	Hierarchical (tree-based)	Partition-based (flat clustering)
Cluster Structure	Dendrogram representing nested clusters	Divides data into k non-overlapping flat clusters
Input Required	No need to specify number of clusters beforehand	Must predefine number of clusters k
Approach	Agglomerative (bottom-up)/Divisive (top-down)	Iterative refinement (assignment + update)
Visualization	Dendrogram (tree structure)	No hierarchical structure is formed
Computational Complexity	Higher: $O(n^2)$ or $O(n^3)$, less efficient for large datasets	Lower: $O(nki)$, faster for large datasets
Flexibility in Shape	Can find arbitrarily shaped clusters	Works best with spherical clusters
Scalability	Poor scalability; suitable for small datasets	Scales well with large datasets
Stability	More stable (doesn't require random initialization)	Sensitive to initial centroid positions
Merge/Split Possibility	Once merged or split, decisions cannot be undone	Centroids and cluster memberships are updated iteratively

DBSCAN (Density-Based Spatial Clustering of Applications with Noise) is a popular **density-based clustering algorithm**. It groups together points that are closely packed (high-density regions) and marks points that lie alone in low-density regions as noise or outliers.

Term	Description
Epsilon (ε)	Radius defining neighborhood around a point
MinPts	Minimum number of points required to form a dense region
Core Point	Point with at least MinPts points in its ε-neighborhood
Border Point	Point that is within ε of a core point but has fewer than MinPts neighbors
Noise Point	Point that is not a core point and not reachable from any core point

Algorithm Steps:
1. **Select an unvisited point** P.
2. **Find all points** within distance ε of P (its ε-neighborhood).
3. **If** the ε-neighborhood contains at least MinPts points, **form a cluster** and mark P as a core point.
4. **Expand the cluster** by recursively adding all density-reachable points (neighbors of neighbors).
5. **If** the ε-neighborhood has fewer than MinPts points, mark P as noise (later it may become a border point).
6. **Repeat** until all points have been visited.

Advantages: Can find clusters of arbitrary shape and size, Automatically detects noise and outliers, No need to specify the number of clusters beforehand.
Disadvantages: Sensitive to parameters ε and MinPts, Difficult to estimate ε for datasets with varying densities, Performance degrades on high-dimensional data.



Clustering Application	Description and Examples
Customer Segmentation	Grouping customers based on buying behavior, demographics, or preferences to tailor marketing strategies.
Image Segmentation	Dividing images into meaningful regions for object detection or recognition (e.g., medical imaging).
Document Clustering	Organizing large text collections into topics or themes, aiding in information retrieval and summarization.
Anomaly Detection	Identifying outliers or fraud in data such as credit card transactions or network security.
Genomics and Bioinformatics	Clustering genes or proteins with similar functions or expressions to understand biological processes.
Market Research	Analyzing survey data to identify groups with similar opinions or behaviors.
Social Network Analysis	Detecting communities or clusters of users based on interaction patterns.
Recommender Systems	Grouping users or items to improve recommendation quality by finding similar preferences.
Earth Sciences	Classifying regions based on soil types, climate, or vegetation for environmental studies.
Health Care	Grouping patients with similar symptoms or diseases for personalized treatment plans.